Name : Sahil Jakhariya
Div : D15C
Batch : C Roll
No : 63

MLDL Experiment 01

Aim: Implement Linear and Logistic Regression on real-world datasets.

Linear Regression:

1. Dataset Source:

- **Dataset Name: Medical Cost Personal Datasets**
- **Source:** Medical Cost Personal Datasets
- **Repository Owner: Miri Choi**

This dataset contains medical insurance records used for practicing regression and machine learning workflows. The data represents realistic relationships between personal attributes and medical insurance charges, making it suitable for exploratory data analysis (EDA), feature engineering, and regression modelling.

- Size: 1338 samples (rows) × multiple attributes (columns)
- Target Variable:

○ charges (Continuous) – Individual medical insurance cost billed by the insurance company.

- **Key Features:**

1. age – Age of the policyholder.

2. sex – Gender of the individual (male or female).
3. bmi – Body Mass Index, indicating body fat based on height and weight.
4. children – Number of dependents covered under insurance.
5. smoker – Indicates whether the individual is a smoker or non-smoker.
6. region – Residential region of the individual (southeast, southwest, northeast, northwest).

The dataset is clean, beginner-friendly, and contains both numerical and categorical features. Minimal preprocessing such as encoding categorical variables is required before applying regression algorithms.

3. Mathematical Formulation of the Algorithm

Multiple Linear Regression models the relationship between several independent variables and a continuous dependent variable. It predicts the medical insurance charges based on input features such as age, BMI, smoking status, and other personal attributes by fitting a linear equation to the observed data.

4. Algorithm Limitations

1. Linearity Assumption: The model assumes a linear relationship between input features and medical insurance charges.
2. Sensitive to Outliers: Extremely high or low medical charges may affect prediction accuracy.
3. Multicollinearity: Highly correlated features such as age and BMI may reduce model stability and affect coefficient interpretation.

5. Methodology / Workflow

1. Load CSV dataset.

2. Select input features and target variable (charges).
3. Convert categorical variables into numerical form using encoding.
4. Split dataset into training and testing sets (80:20).
5. Train Multiple Linear Regression model.
6. Predict medical insurance charges.
7. Evaluate model performance using MSE and R^2 score.

6. Performance Analysis

● Metric 1: Mean Squared Error (MSE)

○ Measures the average squared difference between actual and predicted insurance charges.

● Metric 2: R-Squared (R^2)

○ Indicates the percentage of variance in medical insurance charges explained by the input features.

Higher R^2 and lower MSE indicate better model performance.

7. Hyperparameter Tuning

Standard Linear Regression has no tunable hyperparameters.

To improve performance, Ridge Regression (L2 Regularization) can be applied.

● Tuned Parameter: alpha
● Tested Values: [0.1, 1, 10]

● Helps reduce overfitting and improve model generalization.

Code And Output:

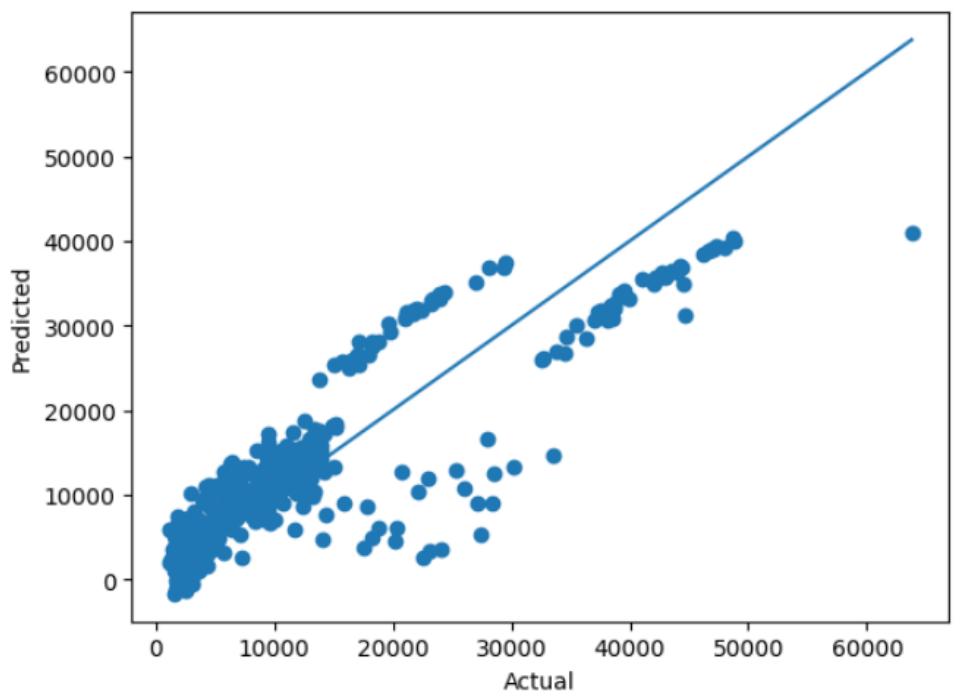
```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression

from sklearn import metrics

df = pd.read_csv('/content/insurance.csv')

df = pd.get_dummies(df, drop_first=True)

X = df.drop('charges', axis=1)

y = df['charges']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = metrics.mean_absolute_error(y_test, y_pred)

mse = metrics.mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = metrics.r2_score(y_test, y_pred)

print("MAE:", mae)
```

```
print("MSE:", mse)

print("RMSE:", rmse)

print("R2:", r2)

plt.scatter(y_test, y_pred)

plt.plot([y.min(), y.max()], [y.min(), y.max()])

plt.xlabel("Actual")

plt.ylabel("Predicted")

plt.show()
```

Choose Files insurance.csv

insurance.csv(text/csv) - 55628 bytes, last modified: 9/27/2019 - 100% done

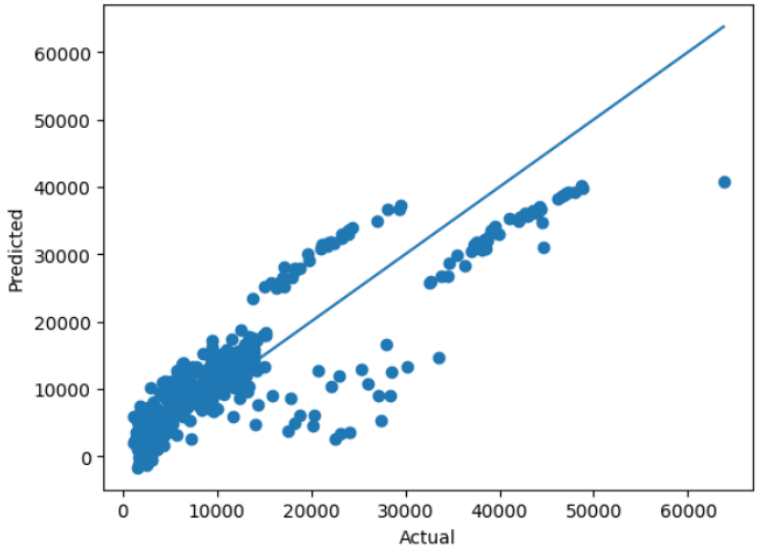
```
Saving insurance.csv to insurance (3).csv
File used: insurance (3).csv
MAE: 4145.450555627587
MSE: 33780509.57479164
RMSE: 5812.100272258871
R2: 0.7696118054369011
```



```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import Ridge
from sklearn import metrics

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

df = pd.get_dummies(df, drop_first=True)

X = df.drop('charges', axis=1)
y = df['charges']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

params = {'alpha': [0.1, 1, 10]}

ridge = Ridge()
grid = GridSearchCV(ridge, params, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)

print("File used:", list(uploaded.keys())[0])
print("Best Alpha:", grid.best_params_)
mae = metrics.mean_absolute_error(y_test, y_pred)
mse = metrics.mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = metrics.r2_score(y_test, y_pred)
print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2:", r2)
plt.scatter(y_test, y_pred)
plt.plot([y.min(), y.max()], [y.min(), y.max()])
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.show()
```

Choose Files insurance.csv

```
insurance.csv(text/csv) - 55628 bytes, last modified: 9/27/2019 - 100% done
Saving insurance.csv to insurance (4).csv
File used: insurance (4).csv
Best Alpha: {'alpha': 1}
MAE: 4158.791710342997
MSE: 33818241.88764884
RMSE: 5815.34537991071
R2: 0.7693544653450765
```



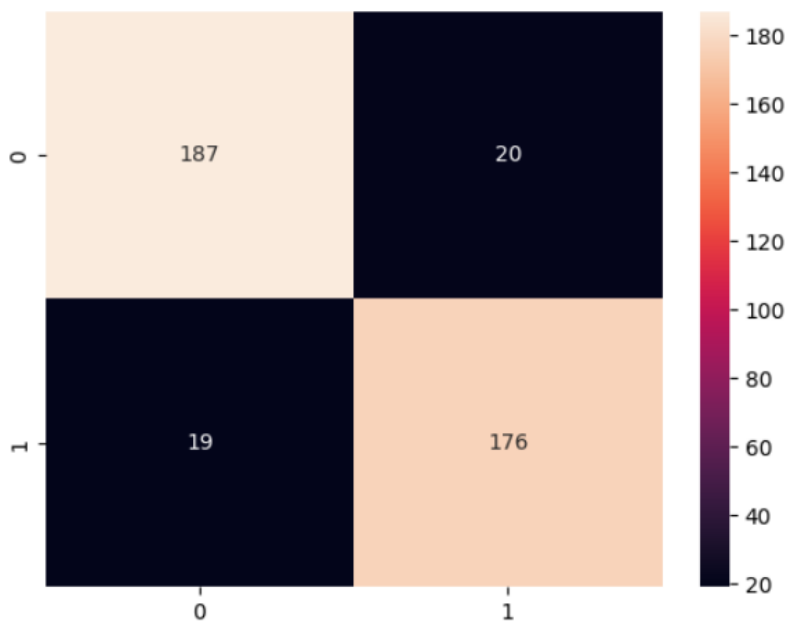

```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

df = pd.get_dummies(df, drop_first=True)

df['target'] = (df['charges'] > df['charges'].median()).astype(int)

X = df.drop(['charges', 'target'], axis=1)
y = df['target']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("File used:", list(uploaded.keys())[0])

acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

Choose Files insurance.csv

insurance.csv(text/csv) - 55628 bytes, last modified: 9/27/2019 - 100% done

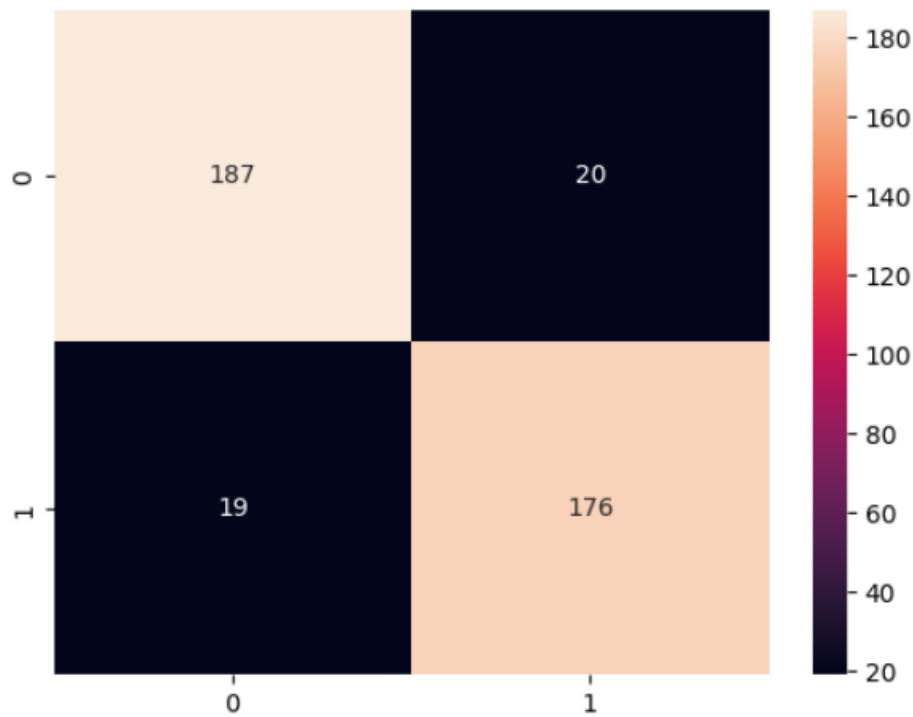
```
Saving insurance.csv to insurance (5).csv
File used: insurance (5).csv
Accuracy: 0.9029850746268657
```



```
from google.colab import files
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix

uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

df = pd.get_dummies(df, drop_first=True)

df['target'] = (df['charges'] > df['charges'].median()).astype(int)

X = df.drop(['charges', 'target'], axis=1)
y = df['target']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

params = {'C': [0.01, 0.1, 1, 10, 100]}

model = LogisticRegression(max_iter=10000)
grid = GridSearchCV(model, params, cv=5)
grid.fit(X_train, y_train)

best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)

print("File used:", list(uploaded.keys())[0])
print("Best C:", grid.best_params_)

acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)

cm = confusion_matrix(y_test, y_pred)
```

```
sns.heatmap(cm, annot=True, fmt='d')
plt.show()
```

```
Choose Files insurance.csv
insurance.csv(text/csv) - 55628 bytes, last modified: 9/27/2019 - 100% done
Saving insurance.csv to insurance (6).csv
File used: insurance (6).csv
Best C: {'C': 1}
Accuracy: 0.9029850746268657
```



Interpretation

Linear Regression

The graph represents the relationship between the actual insurance charges and the predicted charges generated by the Linear Regression model.

- The X-axis represents the actual charges from the test dataset.
- The Y-axis represents the predicted charges produced by the model.
- The diagonal line represents the ideal prediction line, where predicted values perfectly match the actual values.

Most of the data points are reasonably close to the diagonal line, indicating that the model is able to predict insurance charges with moderate accuracy.

The R^2 value (~0.76) indicates that the model explains approximately **76% of the variance** in insurance charges. This shows that the model has learned important relationships between features such as age, BMI, smoking habits, and charges, but not perfectly.

Some data points are scattered away from the diagonal line, indicating prediction errors. This is expected because real-world datasets like insurance data are more complex and contain noise compared to synthetic datasets.

Overall, the Linear Regression model provides a **good but not perfect fit** for predicting insurance charges.

Logistic Regression

The confusion matrix visualizes the classification performance of the Logistic Regression model.
The matrix contains four values:

- **True Positives (TP):** Correctly predicted high insurance charges
- **True Negatives (TN):** Correctly predicted low insurance charges
- **False Positives (FP):** Predicted high charges when actually low
- **False Negatives (FN):** Predicted low charges when actually high

From the matrix values in the experiment:

- **TN = 187**
- **FP = 20**
- **FN = 19**
- **TP = 176**

The model achieves an accuracy of approximately **90%**, indicating strong classification performance.
The confusion matrix shows that the model correctly classifies most cases, with relatively low false predictions. This means the model is effective in distinguishing between high and low insurance charges.

Conclusion

This experiment demonstrated the implementation of Linear Regression for predicting insurance charges and Logistic Regression for classifying insurance costs into high and low categories.
Both models successfully captured relationships between input features and the target variable, showing the effectiveness of supervised learning techniques on real-world datasets.
The experiment also highlighted the importance of:

- Data preprocessing (handling categorical variables)
- Feature scaling (for Logistic Regression)
- Train–test splitting
- Hyperparameter tuning

Performance evaluation using metrics such as **MSE, RMSE, R^2 score, Accuracy, and Confusion Matrix** provided valuable insights into model effectiveness.
Overall, this experiment improved understanding of regression and classification techniques and their application in real-world predictive analytics.